

Haya Project Summary

Project identity: Haya is an Android application for running GGUF language models directly on-device, without depending on a remote inference backend for the core chat experience.

Primary goal: make local large-language-model interaction practical on Android phones by combining a native inference layer, a Kotlin/Compose app shell, and a model discovery/download workflow.

Workspace: E:\Haya\haya | **Report date:** 2026-04-09

1. High-Level Idea

This project is essentially a mobile local-AI client. Instead of sending prompts to a cloud API for every response, it allows the user to import or download a GGUF model and execute inference locally on the Android device itself. The design combines three major concerns:

- **Model execution:** native inference through `llama.cpp` wrapped by the internal `redhat1406` module.
- **User interaction:** a Compose-based chat interface with streaming responses, model switching, and chat history.
- **Model management:** a discovery/recommendation/download layer for browsing runnable Hugging Face GGUF models and selecting the right one for the current device.

In plain terms, Haya behaves like a local ChatGPT-style assistant for Android, but the model lives on the phone and the runtime is tuned around device capability, memory limits, and native library compatibility.

2. Core Product Flow

The main user journey is straightforward:

1. The app starts and checks whether a local GGUF model is already available.
2. If no model is active, the user can browse/download a compatible model or import one manually.
3. The app validates the GGUF file, copies or stores it locally, and loads it through the native runtime.
4. The user opens chat, sends a prompt, and the app streams generated tokens into the chat UI.

5. The user can stop generation mid-response, switch to another model, or continue the conversation with preserved history.

Important product choice: Haya is designed around *offline-first* inference. Network access is used mainly for model discovery and download, not for the main conversational response path.

3. Architecture Overview

Layer	Role	Main Technologies
UI layer	Chat interface, model hub, navigation drawer, state presentation	Kotlin, Jetpack Compose, Material 3
ViewModel / presentation logic	Coordinates chat state, downloads, recommendations, and model activation	ViewModel, StateFlow, coroutines
Domain layer	Model validation, model loading, streaming generation, stop/unload behavior	Kotlin coroutines, flow, synchronization primitives
Data layer	Chat persistence, local model metadata, preferences, HF API access	Room, shared preferences, helper APIs
Native inference layer	Loads GGUF models and generates tokens efficiently on-device	redhat1406 , JNI, C++ , llama.cpp
External model/tooling layer	Model metadata, browsing, and file downloads	Hugging Face APIs, GGUF file mapping, download manager

4. Major Modules

app/ Android application module

This is the user-facing app. It contains the Compose UI, ViewModels, Room database entities and DAOs, domain logic for chat/model loading, and coordination with the model hub.

- **Main entry:** MainActivity.kt
- **Main screens:** chat screen, model hub screen, general settings placeholder
- **State management:** ChatViewModel and ModelHubViewModel
- **Persistence:** chat history and model metadata via Room

redhat1406/ Native LLM wrapper module

This module is the local inference bridge. It decides which native library variant to load depending on CPU capabilities and ABI, reads GGUF metadata, exposes Kotlin methods such as `load`, `getResponseAsFlow`, `benchModel`, and handles generation stop requests.

- **Kotlin façade:** `RedHat1406.kt`
- **GGUF metadata reader:** `GGUFReader.kt` plus JNI bindings
- **Native backend:** C++ + JNI + `llama.cpp`
- **Optimization logic:** different ARM-specific native libraries for device-specific performance

`hf-model-hub-api/` **Hugging Face helper module**

This module handles model search, metadata lookups, and repository tree exploration so the app can identify downloadable GGUF variants and display them in the model hub UI.

`llama.cpp/` **Upstream inference engine submodule**

This is the third-party core inference project used by the native layer. It provides the underlying GGUF model loading and token generation machinery. Haya does not reimplement inference from scratch; it builds on this upstream engine.

5. User Experience Design

The UI is organized around two primary experiences:

- **Model Hub:** discover models, inspect recommendations, download a model, and activate it.
- **Chat:** conduct a conversation using the currently active local model.

The app also includes a sidebar/navigation drawer for switching between model settings, general settings, and chat history. The chat experience is intentionally simplified so the key workflow stays focused: load a model, ask a question, get a streamed response.

6. Chat System Details

The chat subsystem is built to behave like a modern streaming assistant:

- User messages are persisted in the local database.
- Assistant responses stream token-by-token into the UI.
- Current streaming text is stored separately from persisted message history.
- A stop action can interrupt generation while preserving already generated partial output.
- Chat titles are auto-derived from the first user message when the conversation is still untitled.

The recent fix in this workspace addressed an important UX bug where partial streamed text disappeared after pressing Stop. The current behavior preserves the partial response in UI state immediately and also stores it in the database.

7. Model Handling and Safety Logic

One of the more thoughtful parts of the project is its attempt to protect the phone from loading a model that is too heavy for available memory.

- The app validates that the selected file is a non-empty GGUF file.
- It estimates runtime memory requirements based on file size.
- It computes a safe RAM budget from the device profile.
- If the model appears too large, the app blocks the load rather than risking a crash or device instability.
- Previously loaded models can be re-opened on startup.

This makes the app more practical for real phones, where RAM pressure is one of the biggest constraints for local LLM execution.

8. Recommendation and Browsing System

The model recommendation system is more advanced than a simple list of downloads. It tries to match models to both the *device* and the *user's intended use case*.

Inputs include:

- device memory and CPU capability
- model family, quantization, and estimated parameter count
- use case preference such as coding, general chat, or mixed use
- download counts and recency signals

Outputs include:

- a ranked recommendation list
- featured picks such as recommended, fastest, and highest quality
- support estimation and warnings for risky/heavy models

This is a strong product idea because it reduces friction for non-expert users: instead of forcing them to guess which GGUF model their phone can handle, the app helps them choose one intelligently.

9. Native Performance Strategy

The native layer does not rely on a single generic shared library. Instead, it contains multiple prebuilt variants designed for different ARM feature sets:

- baseline ARM64
- ARMv8.2 with FP16
- ARMv8.2 with FP16 + dot product

- ARMv8.4 variants with SVE and/or i8mm where supported
- a 32-bit ARMv7 path

At runtime, the app inspects CPU features from `/proc/cpuinfo` and picks the best matching native library. This is a meaningful optimization because on-device inference performance can vary significantly depending on whether the hardware supports vector instructions, dot-product extensions, and half precision acceleration.

10. Persistence and Local Data Model

The Room database stores:

- **Chats:** title, system prompt, timestamps, model association, context settings
- **Chat messages:** user/assistant messages per chat
- **Models:** saved metadata for downloaded or imported models
- **Tasks/Folders:** supporting structures for broader app organization

Shared preferences store active model identity and preference/onboarding state for recommendation behavior.

11. Technical Strengths

- **Clear product direction:** local AI assistant on Android with real GGUF execution.
- **Practical architecture:** Kotlin app layer cleanly separated from native inference layer.
- **Good UX ambition:** recommendation system reduces model-selection friction.
- **Performance awareness:** ABI and CPU feature-based loading show mobile optimization thinking.
- **Offline-first value proposition:** user privacy and local availability are central advantages.

12. Current Constraints and Weak Spots

- The project still carries substantial native weight because `llama.cpp` and related shared libraries are large.
- The app is simplified relative to a full production assistant, so some settings and broader management flows are still minimal or placeholder.
- There are build-time warnings around KSP/Kotlin version alignment and Room schema export configuration.
- The project depends on correct submodule initialization for `llama.cpp`.
- On-device inference is inherently constrained by RAM, thermal limits, and storage speed on lower-end devices.

13. What This Project Demonstrates

From an engineering and academic perspective, Haya demonstrates a meaningful combination of mobile engineering, native integration, model-management logic, and UI/UX problem-solving.

It is not just “an Android app that calls an AI API.” Instead, it tackles:

- embedding a real local inference runtime
- mobile memory-aware model gating
- device-adaptive model recommendation
- streaming token UX
- local-first persistence and conversation management

That makes it a solid project concept for demonstrating how edge AI can be packaged into a user-facing Android application.

14. Suggested One-Paragraph Executive Summary

Haya is an offline-first Android application that enables users to run GGUF language models locally on their phones. The project combines a Jetpack Compose front end, Room-based local persistence, a Hugging Face-powered model discovery and recommendation layer, and a native JNI/C++ inference wrapper built on top of `llama.cpp`. Its main value lies in making local mobile AI usable: users can import or download a compatible model, load it on-device, chat with it in a streaming interface, and rely on built-in device-aware checks to avoid loading models that are too heavy for their hardware.

15. Conclusion

Overall, this project is a mobile local-LLM platform focused on practical Android deployment. Its strongest idea is the pairing of native on-device inference with a device-aware recommendation and model-management experience. In short, Haya is aiming to make private, offline, phone-native AI interaction both technically feasible and user-friendly.